SEC2_APM1110-SA1_Part2_GROUP_6-DABU, L; MONTILLA, A

GitHub: https://github.com/dabulira20/Prob-ProbDi-FAs/tree/main/SA1

2026-10-11

Contents

Overview **1**

Section II **1**

Analyze the specified Bitcoin and Ethereum return data sets using distribution fitting and normality testing. . 1

Task #1 **3**

Interpretation of The Result 8

Task #2 **8**

Interpretation of The Result 11

Overview

This document contains the complete solution to Section II (Items 1 & 2). Each item includes the code, example output, and a brief interpretation.

Section II

Analyze the specified Bitcoin and Ethereum return data sets using distribution fitting and normality testing.

Settings Preparations

```
set.seed(123)           # makes random sampling repeatable

knitr::opts_chunk$set(echo = TRUE)

library(tidyverse)      # read_csv, dplyr, ggplot2, tibble
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 -
-
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() -
-
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts t
```

```
library(lubridate)      # ymd_hms
library(fitdistrplus)  # fitdist, gofstat, denscomp, qqcomp
```

```
## Loading required package: MASS
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
##
## Loading required package: survival
```

```
btc_file  <- "btcusd_1-min_data.csv"   # path to your Bitcoin file
eth_file  <- "ethusd_1min_ohlc.csv"           # path to your Ethereum file
btc_time_col  <- "Timestamp"          # name of the time column in BTC file
btc_price_col <- "Close"               # name of the price column in BTC
 ↪  file
eth_time_col  <- "timestamp"          # name of the time column in ETH file
eth_price_col <- "close"               # name of the price column in ETH
 ↪  file


# ---- Helper: turn a time column into a real date-time (UTC) ----
to_datetime <- function(x) {
  if (is.numeric(x)) {
    # Unix time: if numbers are huge, they are milliseconds, else seconds
    if (max(x, na.rm = TRUE) > 1e11) x <- x / 1000
    as.POSIXct(x, origin = "1970-01-01", tz = "UTC")
  } else {
    ymd_hms(x, tz = "UTC")    # text such as "2020-01-01 00:01:00"
  }
}

# ---- Helper: log returns, keeping only consecutive time steps ----
# step_sec = 60 for 1-minute data, 300 for 5-minute data
make_returns <- function(df, step_sec) {
  df %>%
    arrange(time) %>%                                  # sort oldest -> newest
    mutate(
```

```
      gap = as.numeric(difftime(time, lag(time), units = "secs")),
      ret = log(price) - log(lag(price))                  # log return
    ) %>%
    filter(!is.na(ret), gap == step_sec,                  # only truly
     ↪  consecutive rows
          is.finite(ret))                                 # drop Inf/-Inf (bad
            ↪  prices)
}
```

Task #1

Find out which probability distribution function best fits Bitcoin's returns for trading data every minute, from January 1, 2012 to September 15, 2026, for Bitcoin quoted in United States dollars or the BTC/USD pair.

```
# 1. Read the data and look at it
btc_raw <- read_csv(btc_file)
```

```
## Rows: 7771846 Columns: 6
## -- Column specification --------------------------------------------------
---------
## Delimiter: ","
## dbl (6): Timestamp, Open, High, Low, Close, Volume
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
names(btc_raw)        # check column names
```

```
## [1] "Timestamp" "Open"      "High"      "Low"       "Close"     "Volume"
```

```
head(btc_raw)         # check the first rows
```

```
## # A tibble: 6 x 6
##    Timestamp  Open  High   Low Close Volume
##        <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
## 1 1325376060  4.58  4.58  4.58  4.58      0
## 2 1325376120  4.58  4.58  4.58  4.58      0
## 3 1325376180  4.58  4.58  4.58  4.58      0
## 4 1325376240  4.58  4.58  4.58  4.58      0
## 5 1325376300  4.58  4.58  4.58  4.58      0
## 6 1325376360  4.58  4.58  4.58  4.58      0
```

```
tail(btc_raw)         # check the last date reaches Sep 15, 2026
```

```
## # A tibble: 6 x 6
##    Timestamp   Open   High    Low  Close  Volume
##        <dbl>  <dbl>  <dbl>  <dbl>  <dbl>   <dbl>
```

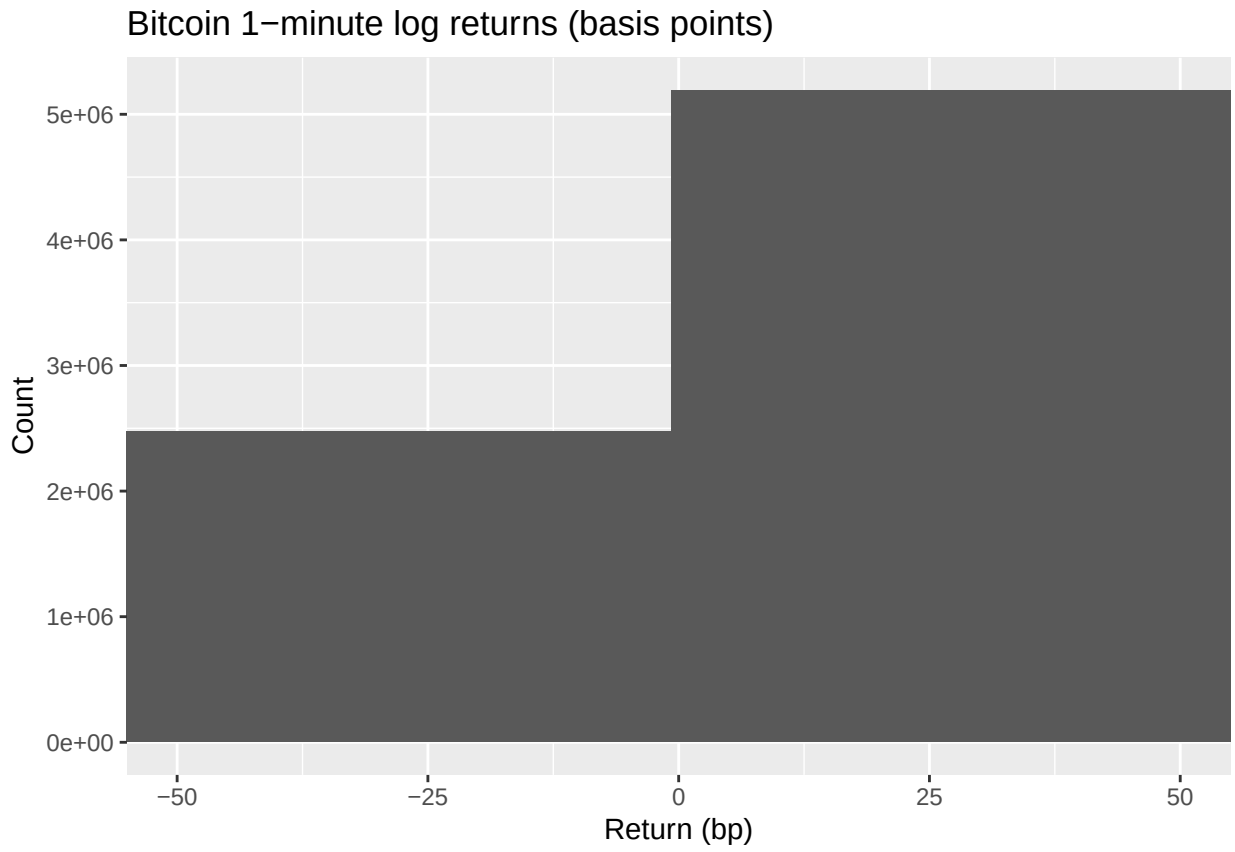
```
## 1 1791686460 83012. 83042. 83012. 83042. 1.10
## 2 1791686520 83042. 83046. 83042. 83046. 0.766
## 3 1791686580 83046. 83055  83046. 83055  1.36
## 4 1791686640 83055  83055  83036. 83038. 0.193
## 5 1791686700 83038. 83048. 83038. 83048. 0.00352
## 6 1791686760 83040. 83040. 83023. 83023. 0.459
```

```
# 2. Build a clean table with just time and price
btc <- tibble(
  time  = to_datetime(btc_raw[[btc_time_col]]),
  price = btc_raw[[btc_price_col]]
) %>%
  filter(!is.na(time), !is.na(price), price > 0) %>%     # remove
   ↪ missing/zero prices
  distinct(time, .keep_all = TRUE) %>%                    # remove duplicate
   ↪ timestamps
  filter(time >= ymd_hms("2012-01-01 00:00:00", tz = "UTC"),
         time <= ymd_hms("2026-09-15 23:59:59", tz = "UTC"))

# 3. Compute 1-minute log returns, in basis points (x 10,000)
btc_ret <- make_returns(btc, step_sec = 60) %>%
  mutate(ret_bp = ret * 10000)
cat("Number of 1-minute returns:", nrow(btc_ret), "\n")
```

```
## Number of 1-minute returns: 7735678
```

```
# 4. Look at the shape
ggplot(btc_ret, aes(x = ret_bp)) +
  geom_histogram(bins = 200) +
  coord_cartesian(xlim = c(-50, 50)) +                    # zoom in on the
   ↪ middle
  labs(title = "Bitcoin 1-minute log returns (basis points)",
       x = "Return (bp)", y = "Count")
```




```r
# 5. Take a random sample so fitting is fast (millions of rows is overkill)
btc_sample <- sample(btc_ret$ret_bp, size = 100000)

# 6. Define two distributions R does not have built in
# Student-t with location (m) and scale (s)
dtls <- function(x, m, s, df) dt((x - m) / s, df) / s
ptls <- function(q, m, s, df) pt((q - m) / s, df)
qtls <- function(p, m, s, df) m + s * qt(p, df)

# Laplace (double exponential) with location (m) and scale (b)
dlaplace <- function(x, m, b) exp(-abs(x - m) / b) / (2 * b)
plaplace <- function(q, m, b) ifelse(q < m, 0.5 * exp((q - m) / b),
                                     1 - 0.5 * exp(-(q - m) / b))
qlaplace <- function(p, m, b) ifelse(p < 0.5, m + b * log(2 * p),
                                     m - b * log(2 * (1 - p)))

# 7. Fit each candidate distribution by maximum likelihood
m0 <- median(btc_sample)            # a good starting guess for the centre
s0 <- mad(btc_sample)               # a good starting guess for the spread

fit_norm  <- fitdist(btc_sample, "norm")
fit_logis <- fitdist(btc_sample, "logis")
fit_cauchy <- fitdist(btc_sample, "cauchy",
                      start = list(location = m0, scale = s0))
fit_tls   <- fitdist(btc_sample, "tls",
                     start = list(m = m0, s = s0, df = 3),
```

```
                      method = "mle", optim.method = "L-BFGS-B",
                      lower = c(-Inf, 1e-6, 0.5))
```

```
## Warning in cov2cor(varcovar): diag(V) had non-positive or NA entries; the
## non-finite result may be dubious
```

```
## Warning in sqrt(diag(varcovar)): NaNs produced
```

```
fit_lap <- fitdist(btc_sample, "laplace",
                   start = list(m = m0, b = mean(abs(btc_sample - m0))),
                   method = "mle", optim.method = "Nelder-Mead")

fits <- list(fit_norm, fit_logis, fit_cauchy, fit_tls, fit_lap)
names_fits <- c("Normal", "Logistic", "Cauchy", "Student-t", "Laplace")

# 8. Compare: lower AIC/BIC = better fit
gof <- gofstat(fits, fitnames = names_fits)
comparison <- tibble(
  Distribution = names_fits,
  AIC = gof$aic,
  BIC = gof$bic
) %>% arrange(AIC)
print(comparison)
```

```
## # A tibble: 5 x 3
##   Distribution     AIC     BIC
##   <chr>          <dbl>   <dbl>
## 1 Student-t    675664. 675693.
## 2 Cauchy       683758. 683777.
## 3 Laplace      718439. 718458.
## 4 Logistic     758625. 758644.
## 5 Normal       843118. 843138.
```

```
cat("Best fit by AIC:", comparison$Distribution[1], "\n")
```

```
## Best fit by AIC: Student-t
```

```
# 9. Visual checks (zoom the density plot into the centre)
bk <- seq(floor(min(btc_sample)) - 1, ceiling(max(btc_sample)) + 1, by =
 ↪  0.5)

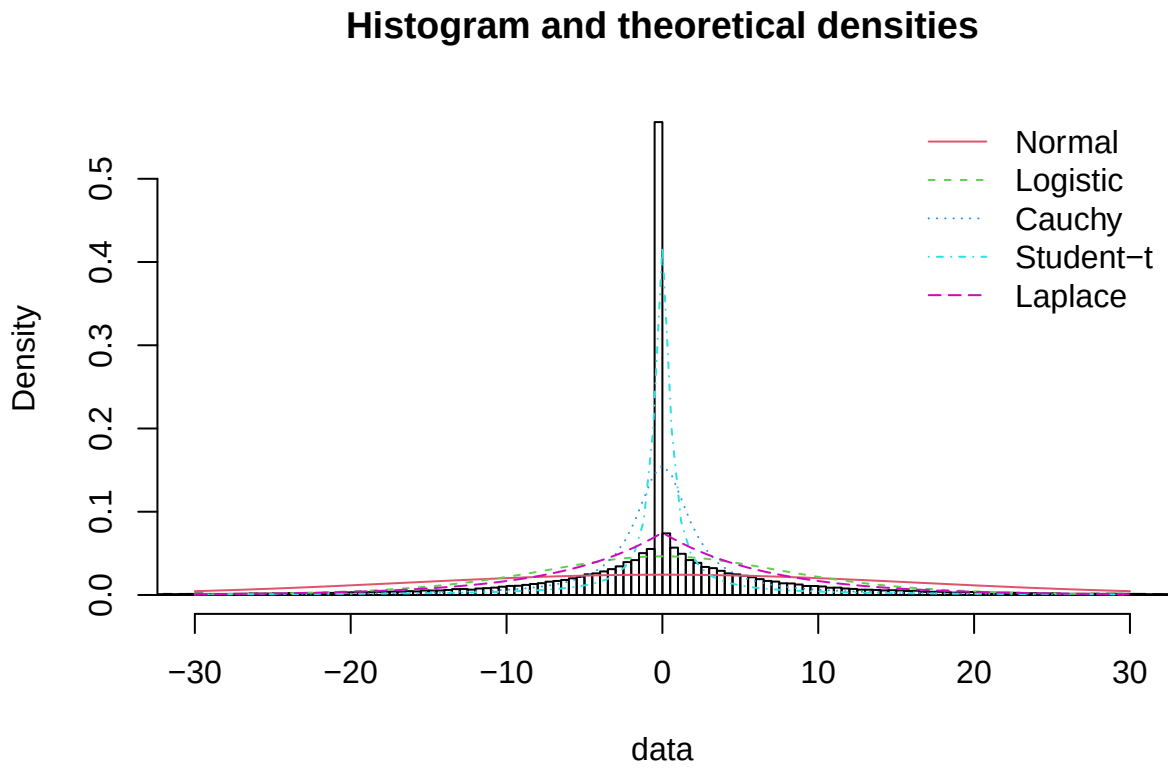
denscomp(fits, xlim = c(-30, 30), breaks = bk, addlegend = FALSE)
```

```
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "breaks" is not a
## graphical parameter
```

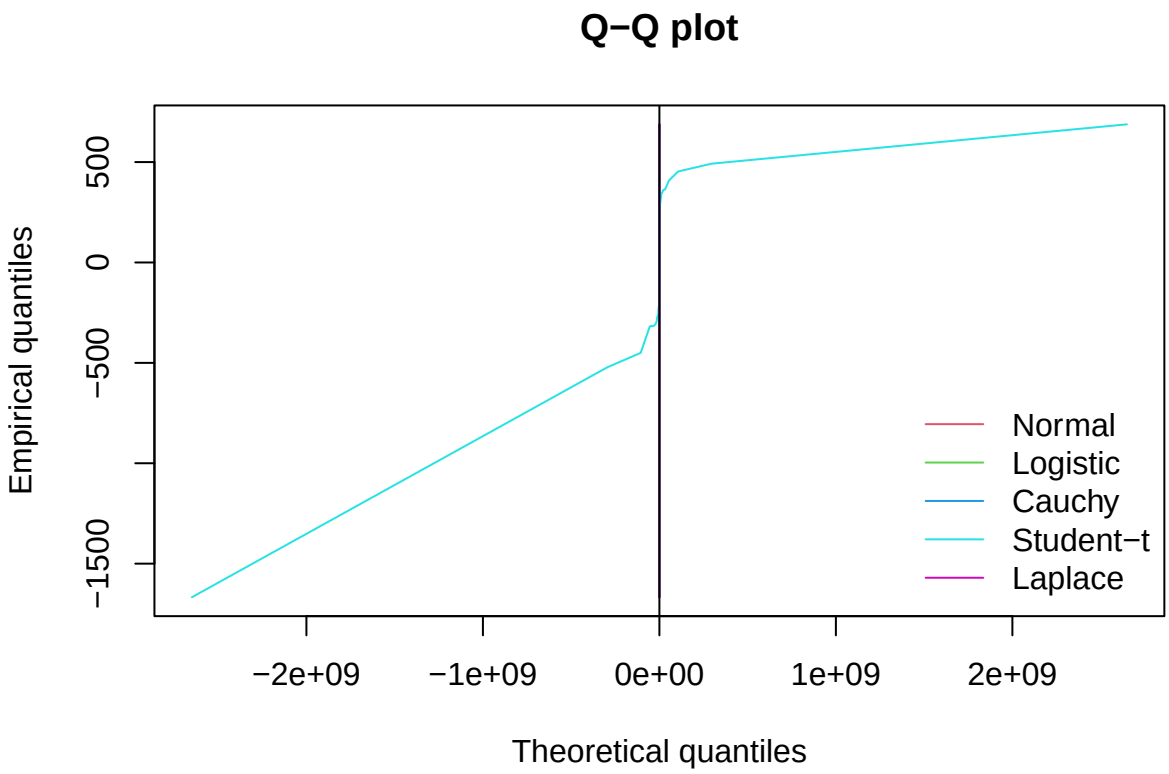
```
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "breaks" is not a
## graphical parameter
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "breaks" is not a
## graphical parameter
```

```
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "breaks" is not a
## graphical parameter
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "breaks" is not a
## graphical parameter
```

```
legend("topright", legend = names_fits, col = 2:6, lty = 1:5, bty = "n")
```




```
qqcomp(fits, legendtext = names_fits)
```



Interpretation of The Result

The Student-t distribution is the best fit for the Bitcoin 1-minute returns, with the lowest values of AIC (675,664) and BIC (675,693) out of the four distributions: Cauchy, Laplace, Logistic, and Normal. Since lower AIC/BIC value means a better trade-off between fit and complexity, and both criteria rank the distributions highest, the conclusion is consistent and not sensitive to the penalty used.

The Normal distribution is the worst fit by a wide margin. Its AIC value is about 167,000 higher than the Student-t's, which is a large gap. This shows that Bitcoin returns are far from normal: they are heavy-tailed, with extreme gains and losses occurring much more often than a Gaussian model allows.

One can also read the ranking for an indication of tail weight. The Student-t and Cauchy, both being heavy-tailed, are the best fit. They are in turn followed by the Laplace and Logistic with their lighter tails, and finally the Normal with the lightest of all. In terms of AIC value the Student-t outperforms the Cauchy by some 8,000 points; the tails are indeed heavy but not to the extent the Cauchy would have it. After all, the Cauchy has infinite variance whereas the Student-t is content with a finite one so long as the degrees of freedom are above 2.

Task #2

Test using Shapiro-Wilk normality test the Ethereum returns for trading data every five minutes, from August 7, 2015 to September 15, 2026.

```r
# 1. Read the data and look at it
eth_raw <- read_csv(eth_file)
```

```
## Rows: 4813098 Columns: 6
## -- Column specification --------------------------------------------------
---------
## Delimiter: ","
## dbl (6): timestamp, open, high, low, close, volume
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
names(eth_raw)
```

```
## [1] "timestamp" "open"      "high"      "low"       "close"     "volume"
```

```r
head(eth_raw)
```

```
## # A tibble: 6 x 6
##    timestamp  open  high   low close volume
##        <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
## 1 1502901900   300   300   300   300   0.02
## 2 1502901960   300   300   300   300   0
## 3 1502902020   300   300   300   300   0
## 4 1502902080   300   300   300   300   0
## 5 1502902140   300   300   300   300   0
## 6 1502902200   300   300   300   300   0
```

```r
tail(eth_raw)         # does it reach Sep 15, 2026? Where does it start?
```

```
## # A tibble: 6 x 6
##    timestamp  open  high   low close  volume
##        <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>
## 1 1791688020 2507. 2507. 2507. 2507.  0.0282
## 2 1791688080 2507. 2507. 2507. 2507.  0
## 3 1791688140 2508. 2508. 2507. 2507.  0.134
## 4 1791688200 2507. 2507. 2505. 2505.  1.31
## 5 1791688260 2505. 2505. 2505. 2505.  1.02
## 6 1791688320 2504. 2504. 2504. 2504. 13.7
```

```r
# 2. Clean table of time and price
eth <- tibble(
  time  = to_datetime(eth_raw[[eth_time_col]]),
  price = eth_raw[[eth_price_col]]
) %>%
  filter(!is.na(time), !is.na(price), price > 0) %>%
  distinct(time, .keep_all = TRUE) %>%
  filter(time >= ymd_hms("2015-08-07 00:00:00", tz = "UTC"),
         time <= ymd_hms("2026-09-15 23:59:59", tz = "UTC"))

cat("Earliest ETH date available:", format(min(eth$time)), "\n")
```

```
## Earliest ETH date available: 2017-08-16 16:45:00
```

```
# 3. Convert 1-minute data to 5-minute data (last price in each 5-minute
 ↪  block)
eth_5min <- eth %>%
  arrange(time) %>%
  mutate(block = floor_date(time, unit = "5 minutes")) %>%
  group_by(block) %>%
  summarise(price = last(price), .groups = "drop") %>%
  rename(time = block)

# 4. Compute 5-minute log returns (300 seconds apart)
eth_ret <- make_returns(eth_5min, step_sec = 300)
cat("Number of 5-minute returns:", nrow(eth_ret), "\n")
```

```
## Number of 5-minute returns: 955382
```

```
# 5. Shapiro-Wilk only accepts 3 to 5000 values, so take a random sample
eth_sample <- sample(eth_ret$ret, size = 5000)

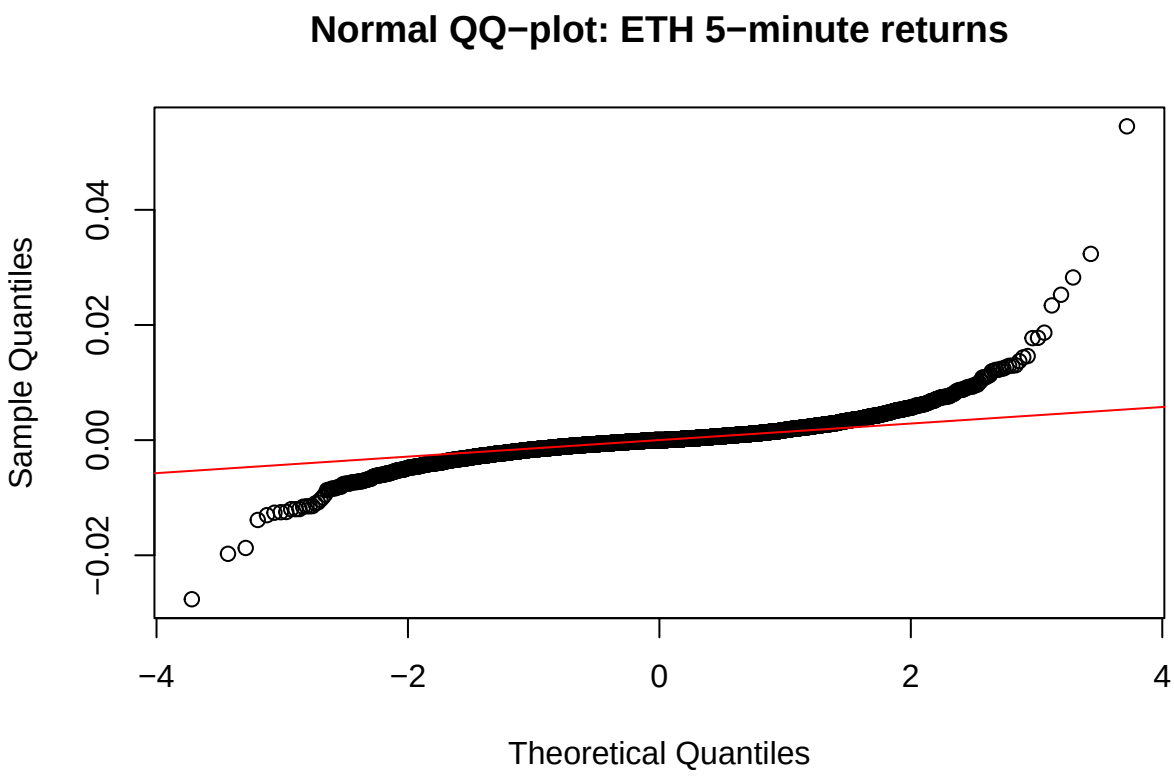
# 6. Run the test
# H0: the returns come from a normal distribution
# H1: they do not
sw <- shapiro.test(eth_sample)
print(sw)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  eth_sample
## W = 0.80154, p-value < 2.2e-16
```

```
# 7. Repeat on 100 different random samples to check the result is stable
p_values <- replicate(100, shapiro.test(sample(eth_ret$ret, 5000))$p.value)
cat("Share of samples rejecting normality at 5%:",
    mean(p_values < 0.05), "\n")
```

```
## Share of samples rejecting normality at 5%: 1
```

```
# 8. Visual check: QQ-plot (points on the line = normal)
qqnorm(eth_sample, main = "Normal QQ-plot: ETH 5-minute returns")
qqline(eth_sample, col = "red")
```




```
# 9. Plain-English decision
if (sw$p.value < 0.05) {
  cat("p < 0.05: reject H0. ETH 5-minute returns are NOT normally
   ↪  distributed.\n")
} else {
  cat("p >= 0.05: fail to reject H0. No evidence against normality.\n")
}
```

```
## p < 0.05: reject H0. ETH 5-minute returns are NOT normally distributed.
```

Interpretation of The Result

For clarification, the dataset contains 4,813,098 one-minute observations from 2017-08-16 16:45:00 to 2026-10-11 03:12:00 (UTC). These were aggregated into 955,382 five-minute returns, and a sample of 5000 returns was used for the Shapiro-Wilk test

A Shapiro-Wilk normality test was run on a sample of 5000 Ethereum 5-minute returns, drawn from the full set of 955,382 returns. The test gave W = 0.80154 and p < 2.2e-16. Because the p-value is far below 0.05, we reject the null hypothesis that the returns are normally distributed.

The W statistic can also draw some conclusions from in and of itself. Since W is bounded at 1, with values near 1 indicating normality, a reading of 0.80 represents a considerable deviation from the norm as opposed to a marginal one. Such a result is in line with what one would expect of high-frequency crypto returns; they are characterised by excess kurtosis in the form of heavy tails and pronounced peaks at zero, meaning extreme price action is far more common than a normal distribution would have it.